Capítulo 7: Herramientas de Git

Actividad 2 - Basado en Pro Git (Chacon & Straub, 2014) - Juan Manuel Gomez Perez - 43972

Introducción

Este capítulo presenta herramientas avanzadas para manejar proyectos complejos: seleccionar confirmaciones, preparar cambios de forma selectiva, guardar trabajo temporal, reescribir historial, depurar y trabajar con submódulos.

7.1 Selección de revisiones

```
git show 1c002dd          # hash abreviado
git log --abbrev-commit --pretty=oneline
git show HEAD~2              # dos confirmaciones antes de HEAD
git show HEAD^               # primer padre
git log main..tema           # en tema pero no en main
git log main...tema          # en una u otra, pero no en ambas
git reflog                 # historial de movimientos de HEAD
```

El reflog es un registro local de dónde ha estado HEAD; es muy útil para recuperar confirmaciones que parecen perdidas.

7.2 Preparación interactiva

```
git add -i
git add -p     # prepara fragmentos (hunks) en lugar de archivos completos
```

Permite dividir cambios de un mismo archivo en varias confirmaciones lógicas.

7.3 Guardado rápido (stash)

```
git stash
git stash list
git stash apply
git stash pop
git stash drop
```

El stash guarda temporalmente los cambios sin confirmar para poder cambiar de rama con el directorio limpio y recuperarlos después.

7.4 Reescribir la historia

```
git commit --amend
git rebase -i HEAD~3    # reordenar, editar, combinar (squash) o eliminar confirmaciones
git filter-branch      # reescritura masiva (existen alternativas más modernas)
```

Reescribir historia solo es seguro mientras las confirmaciones no se hayan compartido.

7.5 Reset al descubierto

git reset mueve HEAD y, según la opción, afecta también al área de preparación y al directorio de trabajo:

- --soft: solo mueve HEAD (los cambios quedan preparados).
- --mixed (por defecto): mueve HEAD y vacía el área de preparación; los cambios quedan en el directorio de trabajo.
- --hard: mueve HEAD y reemplaza todo; descarta los cambios locales. Es destructivo.

git checkout y git restore también pueden trabajar sobre archivos concretos.

7.6 Fusión avanzada

```
git merge --abort
git merge -X ours rama
git checkout --conflict=diff3 archivo
git config --global rerere.enabled true
```

La opción rerere ("reuse recorded resolution") recuerda cómo se resolvió un conflicto para reutilizar la resolución si aparece de nuevo.

7.7 Depuración con Git

```
git blame -L 12,22 archivo.txt   # quién cambió cada línea
git bisect start
git bisect bad
git bisect good v1.0
git bisect reset
git grep "texto"
```

git bisect realiza una búsqueda binaria en el historial para encontrar la confirmación que introdujo un error.

7.8 Submódulos

```
git submodule add <url> libs/libreria
git clone --recurse-submodules <url>
git submodule update --init --recursive
```

Los submódulos permiten incluir otro repositorio dentro del propio, registrando una confirmación específica de ese repositorio.

7.9 Empaquetado, 7.10 Replace y 7.14 Credenciales

git bundle empaqueta datos de un repositorio en un archivo para transferirlos sin red. git replace permite sustituir un objeto por otro sin reescribir el historial. El almacenamiento de credenciales (credential.helper) guarda contraseñas o tokens para no escribirlos en cada operación.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress.
https://git-scm.com/book/es/v2